

Single Sign-On with Multi-Factor Authentication for the SNode.C Network Framework

Jan Nicolas Eberwein, BSc
Digital Media Department Interactive Media
University of Applied Sciences Upper Austria
Softwarepark 11, 4232 Hagenberg im Mühlkreis, Austria
S2410629003@fhooe.at

Abstract

Modern distributed systems increasingly combine browser-based interfaces, backend APIs, and device-facing services. In such environments, authentication and authorization are frequently implemented per component, leading to inconsistent policies, duplicated logic, and a widening attack surface. Single Sign-On (SSO) addresses these problems by centralizing authentication at a dedicated identity component, but this centralization also concentrates risk: a compromised account can provide broad access across the ecosystem. Multi-Factor Authentication (MFA) is therefore an essential counterpart to SSO in practice, especially when administrative functionality, configuration endpoints, or sensitive telemetry are involved.

This paper presents the design and implementation of an SSO solution with integrated MFA for the SNode.C network framework. In contrast to deployments that adopt a third-party identity provider, the presented system is implemented from scratch in C++ as an extension to the SNode.C ecosystem. The solution follows a standards-oriented architecture inspired by OAuth 2.0 and OpenID Connect, using an Authorization Code style flow with Proof Key for Code Exchange (PKCE) and time-based one-time passwords (TOTP) for second-factor verification. The paper derives requirements from the SNode.C context, describes the protocol flow and system architecture, motivates key engineering decisions including cryptographic choices and database modeling, and discusses security properties with respect to a practical threat model. The result is a deployable foundation that improves usability through SSO while raising the security baseline through MFA and strict token validation across services.

Keywords: Single Sign-On, Multi-Factor Authentication, OpenID Connect, OAuth 2.0, PKCE, TOTP, C++, SNode.C

1 Introduction

Authentication is often treated as a local feature of an application, even though it is a system-level concern that shapes trust boundaries, auditability, and incident response. In distributed deployments, each additional web application, dashboard, or service endpoint tends to introduce its own credential handling and session state. Over time, this results in inconsistent password policies, divergent token handling, and duplicated implementation

effort. Beyond maintainability, fragmented authentication amplifies risk: password reuse enables credential stuffing, inconsistent authorization checks lead to privilege escalation, and weak session management increases exposure to replay and token theft.

The SNode.C network framework provides a representative context for these challenges. SNode.C is a C++ network framework designed for event-driven services and layered network applications, where multiple components may expose HTTP endpoints or web-based control surfaces [9]. As the ecosystem grows, users and administrators interact with several services that must share a coherent identity model and access control policy. A central requirement is therefore to decouple identity management from individual services while preserving a simple integration model that fits SNode.C's modular architecture.

Single Sign-On addresses these needs by moving interactive authentication to a dedicated identity provider (IdP) and enabling services to authorize requests using standardized tokens. However, SSO changes the threat profile: the identity component becomes a high-value target and the blast radius of account compromise increases. Multi-Factor Authentication mitigates this by requiring an additional factor during login, reducing the impact of password compromise.

This work contributes a compact SSO+MFA stack implemented directly in C++ as an extension to SNode.C, rather than integrating an external identity platform. The design follows OAuth 2.0 and OpenID Connect concepts to remain compatible with common client patterns and established security guidance [4, 8].

2 Related Work

OAuth 2.0 and OpenID Connect have become the dominant standards for authorization and web-based SSO. Their broad adoption is partly explained by interoperability and by the ability to separate interactive authentication from token-based access control [4, 8]. Nevertheless, both protocols have a non-trivial security surface, and the security of real-world deployments depends on careful selection of flows and correct validation logic.

Formal and empirical analyses have shown that misconfigurations and incomplete checks are common sources of vulnerability. Fett, Küsters and Schmitz provide a comprehensive formal security analysis of OAuth 2.0 and demonstrate how seemingly minor deviations in redirect handling, token binding, or client assumptions can introduce exploitable states [2]. In a complementary analysis, the same authors evaluate OpenID Connect as a web SSO standard and derive concrete security guidelines, including strict validation of issuer metadata and careful handling of state and redirect parameters [3]. These findings motivate a design approach where protocol compliance is treated as a primary engineering objective rather than as an implementation detail.

MFA has evolved from shared-secret second factors, such as one-time passwords, toward stronger, phishing-resistant authenticators. While TOTP remains widely used due to its deployability and minimal infrastructure requirements [6], modern second factors based on public key cryptography aim to reduce phishing and credential relay risks. Lang et al. analyze cryptographic second factors for the modern web and show how security keys can provide a practical and robust second factor at scale [5]. In the context of SNode.C, this motivates a staged approach: implementing deployable MFA first, while structuring the system to support more robust factors in future extensions.

3 System Overview and Requirements

The goal is to provide SSO and MFA that can be integrated into heterogeneous SNode.C services without embedding credential handling into each component. The system is designed around a dedicated IdP that performs interactive authentication and issues tokens, and relying party services that validate tokens at request boundaries. Integration should be incremental: services should gain access control with minimal changes, ideally by attaching middleware that verifies bearer tokens and exposes identity context to application logic.

In addition to the demo protected web application, the design targets protection of SNode.C-based administrative web interfaces in the MQTT suite. Specifically, the MQTTBroker HTTP/HTTPS web interface instances (e.g., ports 8080/8088 on the router) are suited for JWT middleware protection, while MQTT protocol traffic (e.g., port 1883) remains unchanged.

4 Architecture and Protocol Flow

The architecture follows the OAuth 2.0/OIDC split: the IdP handles authentication and token issuance, while relying parties consume tokens to authorize requests [4, 8]. For interactive browser-based clients, an authorization-code style flow is implemented: unauthenticated users are redirected to the IdP, authenticate (optionally with MFA), receive an authorization code, and the client exchanges it for a JWT access token.

Figure 1 summarizes the interactions. The flow uses a state parameter to mitigate CSRF and enforces PKCE to mitigate authorization code interception, including environments where client secrets cannot be protected reliably [7].

Concrete example (demo configuration).

```
GET http://localhost:8084/protected triggers a redirect to the IdP authorization endpoint, e.g.  
http://localhost:8083/oauth2/authorize?response_type=code&client_id=snodec-webapp&  
redirect_uri=http://localhost:8084/callback&scope=openid&state={rand}&code_challenge=  
{pkce}&code_challenge_method=S256.  
After successful login and MFA, the IdP redirects back with code and state. The client  
exchanges the code via POST http://localhost:8083/oauth2/token and receives  
a JWT access token, which is then sent as Authorization: Bearer <token> to the  
protected service.
```

In the current implementation, the flow issues a JWT access token for service authorization; a full OpenID Connect ID token profile can be added as future work.

5 Implementation

The SNode.C framework is extended with an IdP module, a persistence layer for user and flow state, and reusable JWT components (signing, verification, middleware). The IdP exposes endpoints for login, MFA verification and enrollment, authorization requests, and token exchange. The goal is a standards-oriented subset rather than a full identity platform, keeping dependencies and operational complexity aligned with embedded deployments.

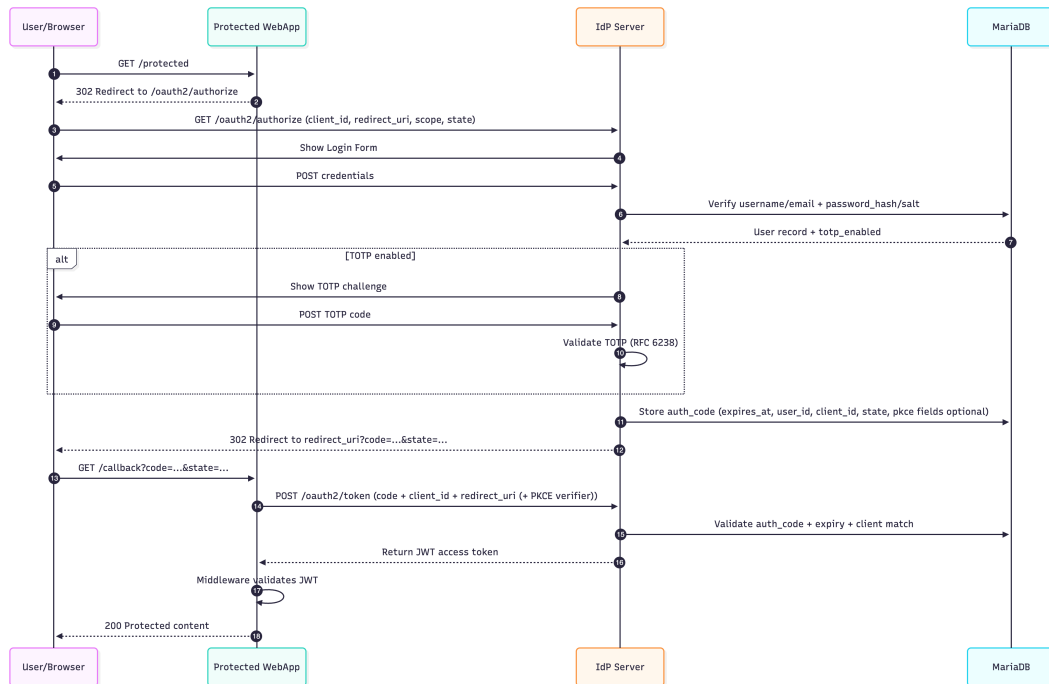


Figure 1: Authentication and authorization flow for the SNode.C identity provider, including TOTP-based MFA, authorization code issuance, token exchange, and subsequent token validation at the protected web application.

5.1 Multi-step Authentication (MFA)

The login process is implemented as a multi-step state machine. After primary credential verification, the IdP checks whether a TOTP second factor is enabled for the account. If enabled, the system requires a valid TOTP code before proceeding. The TOTP implementation follows RFC 6238 defaults (30 s time step, 6 digits, HMAC-SHA1) [6] and uses a drift tolerance of ± 1 step, yielding a 90 s verification window to accommodate minor clock skew.

5.2 Token Issuance and Service Integration

After successful authentication, the IdP issues an authorization code that expires after 10 minutes. Authorization codes are single-use: once exchanged successfully at the token endpoint, the corresponding record is deleted/invalidated to prevent replay. The token endpoint enforces PKCE: `code_verifier` is required and verified against the stored `code_challenge` using S256 (default) or plain. If verification fails, the request is rejected with an OAuth-style error response.

Access tokens are issued as compact JWTs signed with RS256 and a lifetime of 3600 s (1 hour), encoded via the `exp` claim. Relying parties validate the bearer token at request boundaries and only process the request on successful validation.

5.3 Middleware validation rules

```
void middlewareValidateJwt(Request &req, Response &res, Next next) {
    auto auth = req.header("Authorization");
    if (!startsWith(auth, "Bearer ")) { res.status(401); return; }

    auto token = auth.substr(strlen("Bearer "));
    JwtClaims claims; JwtHeader header;

    if (!jwtVerifyAndDecode(token, header, claims)) { res.status(401); return; }
    if (header.alg != "RS256") { res.status(401); return; }

    if (!claims.has("iss") || claims.iss != "http://localhost:8083") {
        res.status(401); return;
    }
    if (!claims.has("aud") || !claims.audienceMatches("snodec-webapp")) {
        res.status(403); return;
    }
    if (!claims.has("exp") || claims.expired()) { res.status(401); return; }

    // Bind token to client context if present
    if (claims.has("azp") && claims.azp != "snodec-webapp") {
        res.status(403); return;
    }

    req.context().set("user_id", claims.sub);
    next();
}
```

Token validation is treated as a security-critical boundary check. Signature verification is combined with strict claim validation to prevent cross-service token reuse and issuer confusion, which are recurring causes of real-world OAuth/OIDC vulnerabilities [2, 3].

5.3.1 Key management

The IdP loads an RSA-2048 private key from `src/auth_idp/keys/private_key.pem` at startup and signs JWTs with RS256. Relying parties validate signatures using the corresponding public key `src/auth_idp/keys/public_key.pem`, distributed to protected services for verification. The JWT header includes a key identifier `kid = snodec-key-2024`, enabling unambiguous key selection and facilitating rotation procedures. A JWKs-style key distribution endpoint is not required for the prototype but is a natural extension for operational deployments.

5.3.2 Redirect URI validation

The IdP enforces strict redirect URI validation using an exact-match allow-list, following OAuth best practices. The allow-list is implemented as a hardcoded set in the current version, including router-local callbacks (e.g., `http://192.168.1.1:8080/callback` and `http://192.168.1.1:8088/callback`) and development callbacks (e.g., `http://localhost:8080/callback`). Authorization requests with a redirect URI outside this set are rejected.

6 Data Model and Persistence

The persistence model supports multi-step authentication and time-bounded flow artifacts. The user table stores password verification material and optional MFA state (TOTP secret and enable flag). A session table supports multi-step flows and includes an expires_at column; in the current prototype the TTL is not configurable and automated cleanup is not implemented. Authorization codes are stored in auth_code with expires_at and PKCE fields (code_challenge, code_challenge_method); access tokens may optionally be persisted for auditability in access_token.

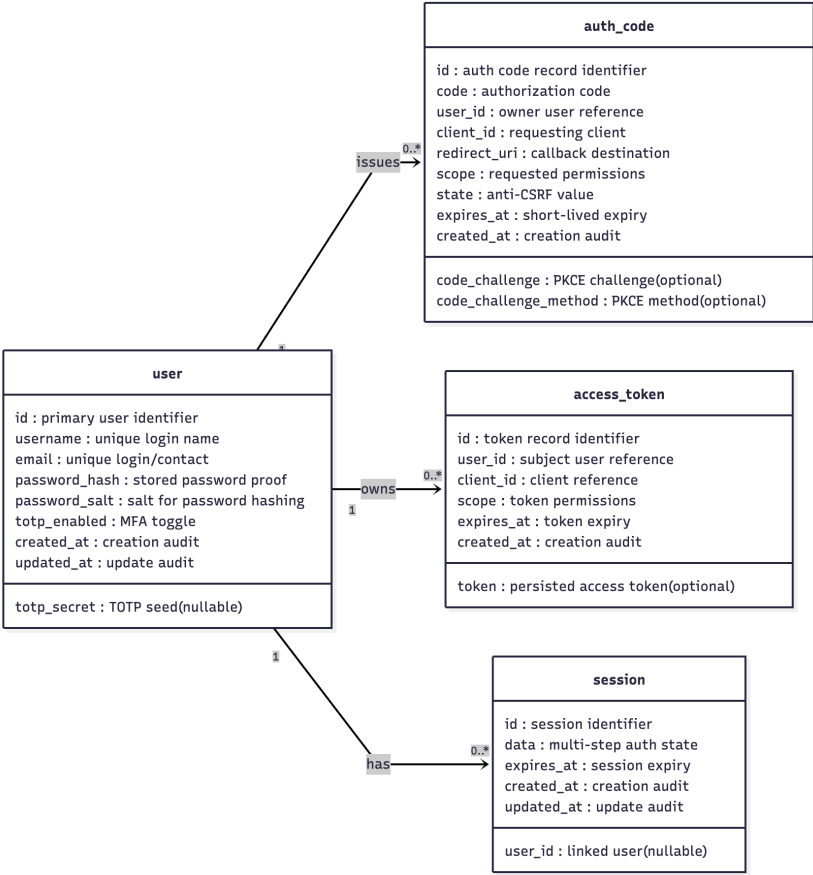


Figure 2: Conceptual database schema supporting multi-step authentication, authorization code issuance, and token lifecycle handling.

Password hashing in the prototype uses SHA-256 with a per-user random salt. This is sufficient for demonstrating correct credential handling and prevents unsalted reuse, but it is not state of the art for password storage. For production deployments with adequate resources, memory-hard hashing such as Argon2id is recommended [1].

7 Security Discussion

The security rationale is shaped by a threat model including leaked passwords, CSRF in browser-based flows, authorization-code interception, and token theft. The implementation maps these threats to concrete checks and invariants, and aligns with security guidance derived from formal analyses of OAuth and OIDC [2, 3].

Table 1: Security invariants and checks at IdP and relying parties.

Component	Mandatory checks / invariants
IdP (/authorize)	Validate <code>redirect_uri</code> by exact allow-list match; require <code>state</code> ; require PKCE parameters (<code>code_challenge</code> , <code>code_challenge_method</code>); enforce MFA completion before issuing an authorization code.
IdP (/token)	Single-use authorization code; code expiry (10 minutes); validate client context; require <code>code_verifier</code> and verify PKCE (S256 default, plain supported); issue short-lived JWTs with <code>exp</code> .
Relying party	Verify signature (RS256) and key selection via <code>kid</code> ; validate <code>iss</code> , <code>sub</code> , <code>aud</code> , <code>exp</code> ; optionally validate <code>azp</code> and <code>iat/nbf</code> when present; reject missing/invalid bearer tokens.

CSRF and code substitution risks are reduced by requiring a state value and rejecting mismatches. Authorization code interception is mitigated by mandatory PKCE [7]; the token endpoint rejects exchanges where the verifier does not match the stored challenge. Redirect-based exfiltration is mitigated by exact-match allow-list validation of `redirect_uri`. Account takeover via leaked passwords is reduced through TOTP-based MFA [6], while recognizing that OTP-based factors remain vulnerable to real-time phishing and relay, motivating future support for phishing-resistant cryptographic authenticators [5].

Token leakage is addressed through time-bounded JWTs (3600 s) and strict service-side validation. Token revocation and refresh tokens are not implemented; security relies on token TTL and re-authentication. This behavior is documented explicitly to set operational expectations.

8 Evaluation

Functional validation covers end-to-end login with and without TOTP, authorization-code issuance and token exchange (including PKCE failure cases), and relying-party rejection behavior for missing, expired, issuer-mismatched, or audience-mismatched tokens.

For performance evaluation, three metrics are most informative: login latency (password + MFA), throughput at `/oauth2/token` (dominated by RSA signing), and per-request middleware overhead (dominated by RSA verification). Table 2 reports the measured latencies and throughput for the login and token flow, covering database queries, SHA-256 password verification, and RS256 (RSA-2048) token signing and verification. Values are reported as ranges across repeated runs under varying concurrency and network conditions; the median and p95 ranges summarize the observed spread.

Table 2: Performance metrics

Metric	Result
Login latency (median)	15–50 ms
Login latency (p95)	80–150 ms
Token endpoint throughput	200–500 req/s
JWT validation overhead	2–5 ms
IdP memory footprint (RSS)	8–20 MB

9 Discussion and Future Work

The presented system balances protocol correctness, deployability, and integration effort within the constraints of a C++ framework. Implementing the IdP directly in SNode.C provides transparent control over authentication behavior, storage, and security-critical validation logic, which is valuable for research and for deployments that prefer self-hosting and minimal external dependencies.

Remaining hardening and feature work is well-scoped: refresh tokens and revocation mechanisms, configurable and enforced session TTL cleanup, rate limiting for credential guessing, automated key distribution (e.g., JWKs), and phishing-resistant authenticators such as security keys [5]. In addition, integrating the JWT middleware into additional SNode.C services (e.g., MQTTBroker web interfaces) provides a direct path toward system-wide consistent authentication policy.

10 Conclusion

This paper presented a standards-oriented SSO solution with integrated MFA implemented directly in C++ as an extension to the SNode.C network framework. The approach applies OAuth 2.0 and OpenID Connect concepts to provide browser-based authentication and token-based authorization for services, while retaining a lightweight integration model aligned with SNode.C’s modular architecture [4, 8, 9]. MFA is supported via TOTP to reduce the likelihood of account takeover through password compromise [6]. The protocol flow was explained with PKCE-enforced authorization code exchange, strict middleware validation (issuer and audience), and a persistence model that supports multi-step authentication and time-bounded flow artifacts. The security discussion links key threats to explicit enforcement points and relates the design to common OAuth/OpenID Connect failure modes reported in prior work [2, 3]. The result is a practical baseline for identity integration in SNode.C deployments and a solid starting point for further hardening of the authentication stack.

References

- [1] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. “Argon2: The Memory-Hard Function for Password Hashing and Other Applications”. In: *2016 IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Ger-*

- many, March 21-24, 2016. IEEE, 2016, pp. 292–302. DOI: 10.1109/EuroSP.2016.31 (cit. on p. 6).
- [2] Daniel Fett, Ralf Küsters, and Guido Schmitz. “A Comprehensive Formal Security Analysis of OAuth 2.0”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’16. ACM, 2016, pp. 1204–1215. DOI: 10.48550/arXiv.1601.01229 (cit. on pp. 2, 5, 7, 8).
- [3] Daniel Fett, Ralf Küsters, and Guido Schmitz. “The Web SSO Standard OpenID Connect: In-depth Formal Security Analysis and Security Guidelines”. In: *30th IEEE Computer Security Foundations Symposium, CSF 2017, Santa Barbara, CA, USA, August 21-25, 2017*. IEEE Computer Society, 2017, pp. 189–202. DOI: 10.1109/CSF.2017.20 (cit. on pp. 2, 5, 7, 8).
- [4] Dick Hardt. *The OAuth 2.0 Authorization Framework*. Oct. 2012. DOI: 10.17487/RFC6749 (cit. on pp. 2, 3, 8).
- [5] Juan Lang et al. “Security Keys: Practical Cryptographic Second Factors for the Modern Web”. In: *Financial Cryptography and Data Security (FC 2016)*. Vol. 9603. Lecture Notes in Computer Science. Springer, 2016, pp. 422–440. DOI: 10.1007/978-3-662-54970-4_25 (cit. on pp. 2, 7, 8).
- [6] David M’Raihi et al. *TOTP: Time-Based One-Time Password Algorithm*. May 2011. DOI: 10.17487/RFC6238 (cit. on pp. 2, 4, 7, 8).
- [7] Nat Sakimura, John Bradley, and Narendra Nandakumar. *Proof Key for Code Exchange by OAuth Public Clients*. Sept. 2015. DOI: 10.17487/RFC7636 (cit. on pp. 3, 7).
- [8] Nat Sakimura et al. *OpenID Connect Core 1.0 incorporating errata set 2*. Accessed: 2025-12-04. 2014. URL: https://openid.net/specs/openid-connect-core-1_0.html (cit. on pp. 2, 3, 8).
- [9] Volker Christian. *SNode.C: A Network Framework in C++*. Accessed: 2025-12-19. URL: <https://github.com/SNodeC/snode.c> (cit. on pp. 2, 8).